

Second-Order System Response

Damping ratio and natural frequency set the whole transient.

Ogata, *Modern Control Engineering*, Ch. 5.

In this tutorial you will:

- sweep the damping ratio ζ and watch the response change,
- compute rise time, peak time, overshoot, and settling time, and
- check the closed-form specs against `stepinfo`.

Step through with **Ctrl+Enter**, or render a report with **publish**.

The standard second-order transfer function is

$$\frac{Y(s)}{R(s)} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

where ω_n is the **undamped natural frequency** and ζ is the **damping ratio**. The closed-loop poles are

$$s = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$$

Depending on ζ , the system is:

- **Overdamped** ($\zeta > 1$): two distinct real poles, no overshoot.
- **Critically damped** ($\zeta = 1$): repeated real pole at $-\omega_n$.
- **Underdamped** ($0 < \zeta < 1$): complex-conjugate poles, decaying oscillation.
- **Undamped** ($\zeta = 0$): poles on the $j\omega$ -axis, sustained oscillation at ω_n .

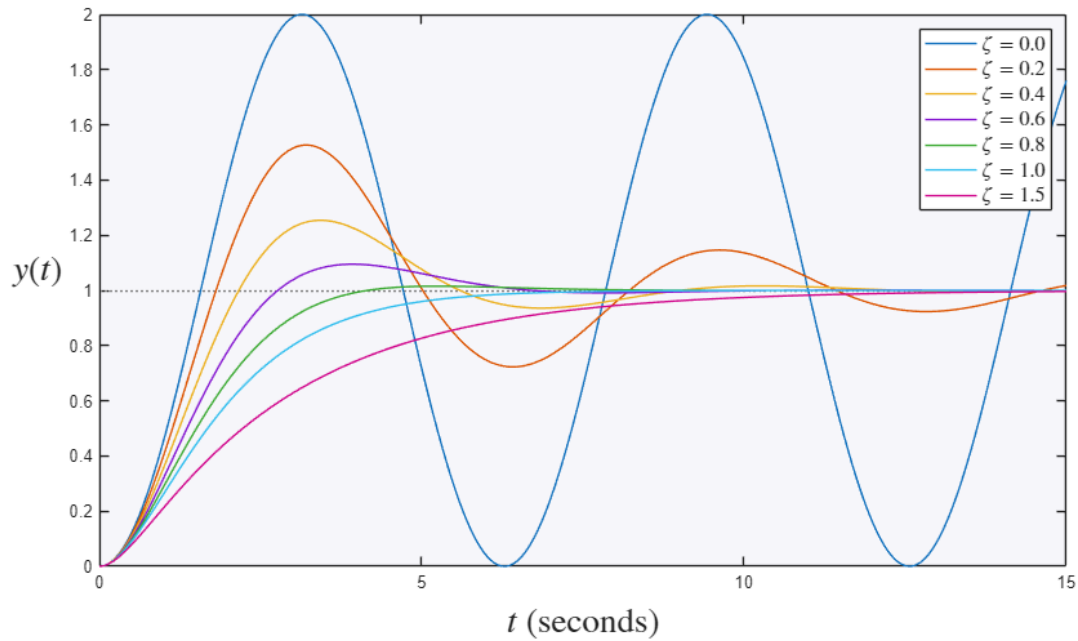
Contents

- Family of Step Responses for Varying ζ
- Underdamped Transient-Response Specifications
- Verification via MATLAB's `stepinfo`
- What Changes with Damping: Pole Locations
- Try it yourself

Family of Step Responses for Varying ζ

```
wn = 1;
zetas = [0 0.2 0.4 0.6 0.8 1.0 1.5];
figure
hold on
for z = zetas
    Gz = tf(wn^2,[1 2*z*wn wn^2]);
    step(Gz, 0:0.01:15)
end
hold off
legend(arrayfun(@(z) sprintf('\zeta=%.1f$',z), zetas, 'UniformOutput', false), ...
    'Interpreter','latex','FontSize',12)
title('Second-Order Step Response vs. Damping Ratio','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter','latex','FontSize',20)
```

Second-Order Step Response vs. Damping Ratio



Underdamped Transient-Response Specifications

For the underdamped case ($0 < \zeta < 1$), the unit-step response is

$$y(t) = 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}} \sin(\omega_d t + \beta), \quad \omega_d = \omega_n \sqrt{1-\zeta^2}, \quad \beta = \cos^{-1} \zeta$$

From this, the standard transient specifications are derived:

- **Rise time** (0% to 100%): $t_r \approx \frac{\pi - \beta}{\omega_d}$
- **Peak time**: $t_p = \frac{\pi}{\omega_d}$
- **Maximum overshoot**: $M_p = e^{-\zeta\pi/\sqrt{1-\zeta^2}}$ (fraction of final value)
- **2% Settling time**: $t_s \approx \frac{4}{\zeta\omega_n}$

```
zeta = 0.5; wn = 4;
G = tf(wn^2,[1 2*zeta*wn wn^2]);

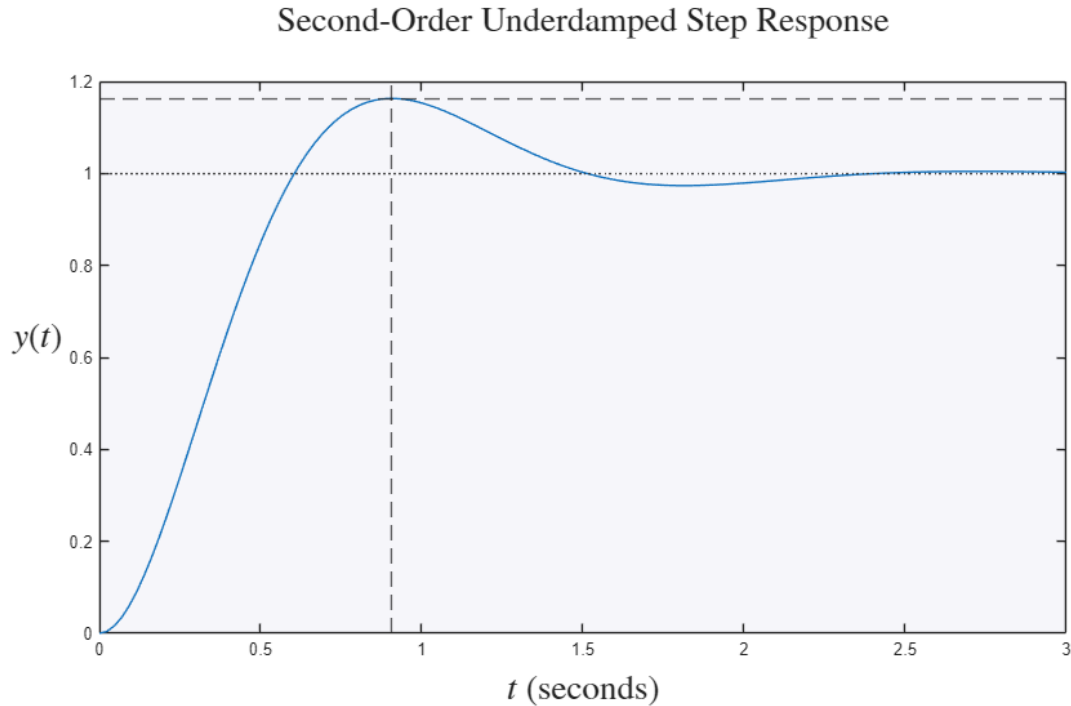
wd = wn*sqrt(1-zeta^2);
beta = acos(zeta);
tr = (pi-beta)/wd;
tp = pi/wd;
Mp = exp(-zeta*pi/sqrt(1-zeta^2));
ts = 4/(zeta*wn);

fprintf('Rise time      tr = %.4f s\n', tr)
fprintf('Peak time      tp = %.4f s\n', tp)
fprintf('Max overshoot  Mp = %.2f%%\n', Mp*100)
fprintf('Settling time  ts = %.4f s (2%% criterion)\n', ts)

figure
step(G)
hold on
yline(1+Mp,'--')
yline(1,':')
xline(tp,'--')
hold off
title('Second-Order Underdamped Step Response','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
```

```
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)
```

```
Rise time      tr = 0.6046 s
Peak time      tp = 0.9069 s
Max overshoot  Mp = 16.30%
Settling time  ts = 2.0000 s (2% criterion)
```



Verification via MATLAB's stepinfo

The Control System Toolbox's `stepinfo` computes these same specifications numerically from the simulated response, providing a check on the closed-form formulas above.

```
info = stepinfo(G)
```

```
info =
```

```
struct with fields:
```

```
    RiseTime: 0.4098
TransientTime: 2.0190
SettlingTime: 2.0190
SettlingMin: 0.9315
SettlingMax: 1.1629
    Overshoot: 16.2929
    Undershoot: 0
        Peak: 1.1629
    PeakTime: 0.8980
```

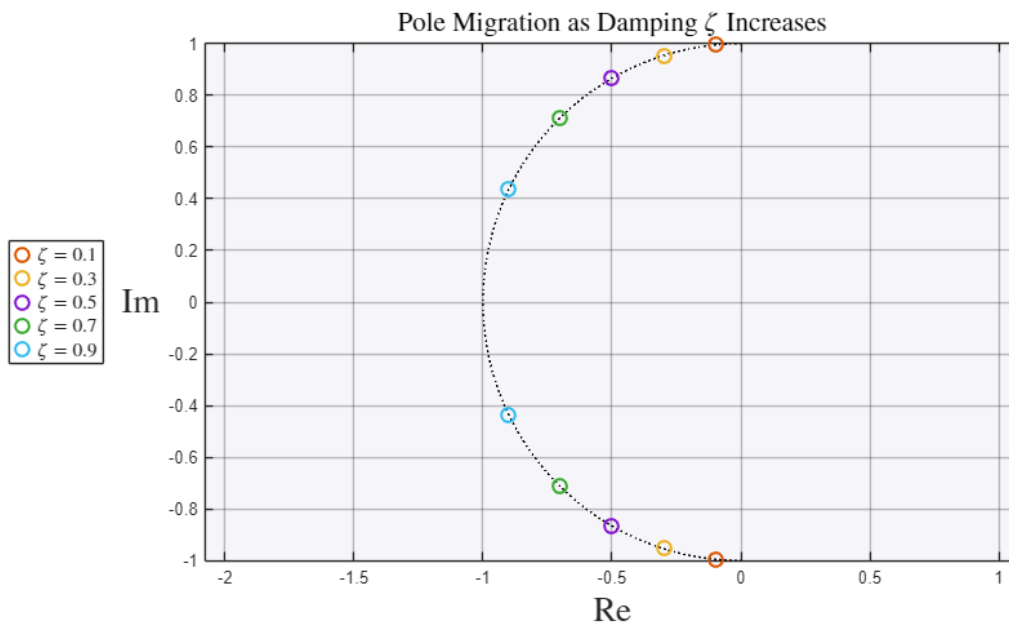
What Changes with Damping: Pole Locations

The family of step responses at the top is driven by where the poles sit. At fixed ω_n , increasing ζ slides the complex pair along a circle of radius ω_n – from the imaginary axis (undamped, sustained ringing) toward the negative real axis (over-damped, no overshoot). This is the "before/after" of adding damping.

```

wn_fixed = 1;
zetas_pm = [0.1 0.3 0.5 0.7 0.9];
figure
th = linspace(pi/2,3*pi/2,200);
plot(wn_fixed*cos(th),wn_fixed*sin(th),'k:','HandleVisibility','off')
hold on
for z = zetas_pm
    p = roots([1 2*z*wn_fixed wn_fixed^2]);
    plot(real(p),imag(p),'o','MarkerSize',8,'LineWidth',1.5, ...
        'DisplayName',sprintf('$\zeta=%.1f$',z))
end
hold off
grid on; axis equal
legend('Interpreter','latex','FontSize',11,'Location','westoutside')
title('Pole Migration as Damping  $\zeta$  Increases','Interpreter','latex','FontSize',16)
ylabel('$\mathrm{Im}$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$\mathrm{Re}$','Interpreter','latex','FontSize',20)

```



Try it yourself

- Set $\zeta = 0.1$ and notice the huge overshoot and long ringing; then try $\zeta = 1.2$ for a sluggish, overshoot-free response.
- Hold ζ fixed and double ω_n : notice the shape stays the same but the time axis compresses (faster, same overshoot).